

Database Project Analysis Report

Group 1

Group Members: Rundong Luo, Yuchen Shi, Rachel Te, Itgel Enkh-Amgalan

Date: May 11, 2026

Introduction

This project designs and implements a database-driven information system for Last Resort Hotel, a large hotel complex that manages sleeping rooms, suites, meeting rooms, reservations, guests, events, billing, room status, and facility usage. The original business description presents a realistic hotel operation with many connected responsibilities: staff must know which rooms are available, which guests are assigned to which rooms, which events are using which facilities, who is responsible for payment, and how guest activity and service charges should be tracked.

Our goal was to convert this complex organizational description into a normalized relational database and a Flask-based web interface that supports operational decision-making. The final system uses SQLite as the database backend and includes 36 database tables, populated sample data, and multiple query-driven pages for rooms, reservations, operations, billing, and guest information.

The design focuses on the core information needs of the hotel: room availability, guest stays, event scheduling, billing responsibility, charge tracking, access logging, guest communication, and staff access to current operational information.

System Overview

Last Resort Hotel contains multiple buildings, wings, floors, and rooms. Rooms may serve different purposes, including sleeping rooms, suites, meeting rooms, and rooms with multiple possible capabilities. Guests may stay overnight, attend events, belong to organizations, receive messages, and use facility access cards. Events may require one or more rooms, may involve multiple guests or organizations, and may generate separate master bills. Billing can be handled by individual guests, outside parties, or organizations.

The final ERD divides the system into the following major modules:

- Hotel Structure: hotel, building, wing, level, and room.
- Room Features: capability_type, room_capability, fixture_type, room_fixture, bed_type, room_bed, adjacent, and room_status.
- Reservation & Stay: reservation, reservation_preference, reservation_bed_preference, stay_room_assignment, stay_room_guest, and room_extension.
- People & Organizations: person, guest, employee, organization, organization_member, and party.
- Events: event, event_guest, event_organization, and event_room_usage.
- Billing: account, account_responsibility, charge, and payment.
- Access & Logs: reader, card_swipe_log, and guest_message.

This modular structure makes the database easier to understand and maintain. Each module handles one part of the hotel operation, while foreign keys connect the modules into a complete information system.

Key Database Design Decisions

Hotel Structure: We separated the physical hotel hierarchy into hotel, building, wing, level, and room. This reflects the original description, where a hotel complex may contain multiple buildings, each building may contain wings, and each wing may contain multiple floors and rooms. This design also supports room numbers that are meaningful within a location rather than globally unique across the whole hotel.

Room Features: A major design decision was to separate physical rooms from room capabilities. A room is stored once in the room table, while its possible uses are stored in room_capability. This allows one physical room to function as a sleeping room, suite, meeting room, or multiple-use room without duplicating the room record. Beds and fixtures are also normalized through room_bed and room_fixture so that rooms can have different combinations of beds, toilets, meeting space, or outdoor space.

Reservation & Stay: Reservations are separated from actual room assignments. The reservation table stores the customer's intended stay dates and billing information, while stay_room_assignment stores the specific room assigned. This is important because the description states that exact room assignments may not be made until close to arrival and may change because of repairs, renovations, cleaning delays, or guest requests.

People & Organizations: We separated person, guest, employee, organization, organization_member, and party. The person table stores basic human contact

information. A person may become a guest or employee through role-specific tables. The party table provides a unified billing identity for either an individual person or an organization, which supports the business rule that the user of a facility and the responsible payer may be different.

Events: Events are modeled separately from reservations because an event may involve hosts, guests, organizations, rooms, and billing even when it is not simply a hotel stay. The event_room_usage table records the room, usage date, usage time slot, and whether the room can be used for eating or not. This reflects the meeting-room scheduling model described in the prompt.

Billing: Billing was modeled through accounts rather than attaching all charges directly to reservations or events. Each account is linked to either a reservation or an event, and charges and payments are linked to accounts. The account_responsibility table records which party is responsible for payment and whether that responsibility is full, split, or backup. This supports master bills, split billing, separate incidentals, event bills, and open balances.

Access & Logs: The reader and card_swipe_log tables allow the system to record guest or employee activity at room, meeting room, entry, and facility readers. The guest_message table supports the business requirement that staff may need to contact guests or record guest-provided availability information.

The ERD was designed to account for the main requirements in the project description. The following examples show how important business details were represented in the database:

- Hotel hierarchy and room numbering are represented through hotel, building, wing, level, and room.
- Room functions such as sleeping, suite, and meeting use are represented through capability_type and room_capability.
- Beds, fixtures, toilets, extra meeting space, and outdoor space are represented through bed_type, room_bed, fixture_type, and room_fixture.
- Adjacent rooms and private or movable-wall doors are represented through adjacent.
- Cleaning, renovation, reconstruction, occupancy, and out-of-service periods are represented through room_status.
- Customer preferences for guest count, smoking, proximity, and bed type are represented through reservation_preference and reservation_bed_preference.
- Actual room assignment and room changes during a visit are represented through stay_room_assignment and stay_room_guest.

- Events, hosts, attendees, organizations, room usage, and time slots are represented through event, event_guest, event_organization, and event_room_usage.
- Individual, outside, and organizational billing responsibility is represented through party, account, account_responsibility, charge, and payment.
- Guest location, staff access records, and guest messages are represented through reader, card_swipe_log, and guest_message.

Assumptions and Ambiguities

The project description includes several ambiguities, so we made explicit assumptions to keep the design coherent and implementable.

First, we assumed that each room belongs to exactly one level, each level belongs to one wing, each wing belongs to one building, and each building belongs to one hotel. Room numbers are unique within their location rather than globally unique across the entire hotel.

Second, we assumed that room smoking status can be determined by the floor or room. The level table stores whether a floor is smoking, while room includes an optional smoking override. This allows an entire floor to be marked smoking or nonsmoking while still permitting exceptions.

Third, we assumed that room status should be time-based. Instead of storing only a current status on the room table, room_status records include start and end dates. This allows the system to track historical, current, and future maintenance, cleaning, renovation, reconstruction, and occupancy blocks.

Fourth, we assumed that a reservation may exist before a specific room assignment is made. Reservation preferences are stored separately from room assignments so the system can first record customer needs and later assign a specific room.

Fifth, we assumed that billing responsibility should be handled at the account level. A reservation has a booked-by party and billing party, but detailed payment information is stored in account_responsibility. This allows a stay to have multiple accounts or split responsibility.

Sixth, we assumed that card readers can represent rooms, meeting rooms, entries, and general facilities. A card swipe log stores either a guest or an employee swipe, but not both at the same time. This supports both guest location tracking and employee operational activity.

Finally, we assumed that the web interface should focus on operational clarity rather than complete production automation. The interface demonstrates the most important information flows without attempting to implement every possible hotel policy.

Normalization and Constraints

The database was designed to follow Third Normal Form. Repeating groups were separated into linking tables, and each table stores facts about one main entity or relationship.

Examples include:

1. Room beds are stored in room_bed instead of storing multiple bed fields directly on room.
2. Room capabilities are stored in room_capability instead of using many boolean columns on room.
3. Guest-event relationships are stored in event_guest.
4. Organization-event relationships are stored in event_organization.
5. Account responsibility is stored in account_responsibility so multiple parties can be linked to one account.
6. Reservation bed preferences are stored in reservation_bed_preference.

The schema also uses primary keys, foreign keys, unique constraints, and check constraints. For example, reservation checkout dates must be after check-in dates, room status end dates must be after start dates, charge amounts must be nonnegative, payment amounts must be positive, and status values are restricted to defined categories. These constraints help preserve database integrity and reduce invalid records.

Data Population Strategy

The final database includes realistic sample data across several months of hotel activity, from May 2026 through September 2026. The populated data includes:

- 20 rooms
- 21 reservations
- 21 stay room assignments
- 14 events
- 28 accounts

- 75 charge records
- 24 payment records
- 33 room status records
- 30 card swipe logs
- 17 guest messages
- 18 guests
- 5 organizations

The data was designed to support meaningful queries rather than only demonstrate table structure. It includes checked-out stays, checked-in stays, future reservations, cancelled reservations, open and closed accounts, room movements, maintenance blocks, event schedules, organization-hosted events, split responsibility, guest messages, and card reader activity.

Queries and Interpretation

The web application uses multiple queries that combine joins, grouping, filtering, aggregation, and date logic. These queries turn raw records into useful hotel information.

- **Current hotel summary:** The home page counts current guests, upcoming events, open accounts, and maintenance rooms. This gives staff a quick operational snapshot before they move into detailed pages.
- **Current guests and affiliations:** The operations page joins room assignments, guests, people, rooms, wings, organizations, and upcoming events. This helps staff identify who is currently in the hotel, where they are assigned, and whether they are connected to an event or organization.
- **Active and upcoming events:** The operations page lists events in the current window, their hosts, scheduled rooms, and room usage slots. This supports event coordination and room preparation.
- **Event guest count comparison:** The system compares linked event guests with estimated staying guests. This helps staff identify whether event-related guest information is complete.
- **Room availability search:** The rooms page filters available sleeping rooms and suites by date range. It excludes rooms blocked by active assignments or unavailable room statuses such as occupied, dirty, renovation, reconstruction, or out of service.
- **Room status summary:** The rooms page groups rooms by status for a selected date. This helps housekeeping and maintenance staff see how many rooms are

ready, occupied, dirty, or blocked.

- Reservation trend analysis: The rooms page groups reservations by month and calculates reservation count, room nights, and expected guests. This helps management understand demand over time.
- Account balance calculation: The billing page aggregates charges and payments for each account, then calculates balance due. This helps staff identify open balances and verify account responsibility.
- Revenue breakdown: The billing page groups charges by charge type and calculates total amount, number of charge lines, average charge amount, and percentage of total revenue. This shows which hotel services generate the most revenue.
- Guest contact and location query: The guests page uses card swipe logs and guest messages to show the latest known location and latest message for each guest, while respecting confidential guest status.
- Guest service usage: The guests page summarizes guest-specific service charges by charge type, count, total amount, and most recent use. This helps staff understand a guest's current relationship with the hotel.

These queries go beyond simple row listings. They answer practical hotel questions about availability, workload, billing, event planning, guest contact, and revenue.

Web Interface

The web interface is implemented with Flask and SQLite. It includes six main pages: Home, Operations, Reservations, Rooms, Billing, and Guests.

The Home page provides a high-level dashboard with current guests, upcoming events, open accounts, and maintenance rooms.

The Operations page focuses on current guests and events. It allows users to create, update, and delete event records, link guests to events, and assign rooms and time slots to events.

The Reservations page allows users to search reservations, create and update reservations, manage stay room assignments, and link guests to room assignments.

The Rooms page displays room availability, room status summaries, reservation trends, and room status records. It also allows staff to add or update room status information.

The Billing page displays accounts, responsible parties, charges, payments, revenue

breakdown, and account balances. It supports adding accounts, charges, and payments.

The Guests page manages guest records, guest confidentiality, messages, latest known location, and guest-specific service usage.

The interface includes more CRUD functionality than the minimum requirement, but its main purpose remains query presentation. Each page is organized around operational questions that hotel staff would realistically need to answer.

Tradeoffs and Limitations

One tradeoff in the design is that some hotel policies are represented through data and application logic rather than through database triggers. For example, room assignment conflicts are checked in the Flask application when creating or updating assignments. A more advanced version could enforce these rules through database triggers.

Another limitation is that the system stores room adjacency but does not include a full graphical room layout. The original description mentions the possibility of graphical room views and zooming between adjacent rooms. Our ERD stores adjacency information, but the web interface displays room data in tables rather than a visual floor plan.

The meeting room pricing rules are also simplified. The schema stores eating and noneating usage slots and charge records, but the system does not automatically calculate all discounts, free noneating slots, or waived charges based on event guest counts. Instead, actual charges are recorded in the charge table.

The ERD also represents adjacency and room capabilities, but it does not fully automate complex movable-wall subdivision scheduling. A future version could add more detailed subroom assignment rules for large divisible meeting rooms.

The system does not implement user authentication or role-based access control. This matches the project requirement that authentication is not necessary, but a production hotel system would need staff accounts, permissions, and stronger audit logging.

Finally, the sample dataset is realistic enough to support queries, but it is still smaller than a real hotel chain database. A real deployment would require more rooms, more buildings, more guests, more events, and stronger performance considerations.

Team Work Division

1. Rundong Luo: first ERD design, final analysis report, final presentation slides, and queries and interpretation.
2. Yuchen Shi: first ERD design, web implementation, mockup website sample, and final presentation slides.
3. Rachel Te: draft analysis report, final presentation slides, and queries and interpretation.
4. Itgel Enkh-Amgalan: final ERD design, final presentation slides, and draft analysis report.